Name: Saai Vardhan D
Registration number : 192525205

CSA 0421 - OPERATING SYSTEM

Experiment 21

Develop a C program to implement worst fit algorithm of memory management.

Executable C code :

```
#include <stdio.h>

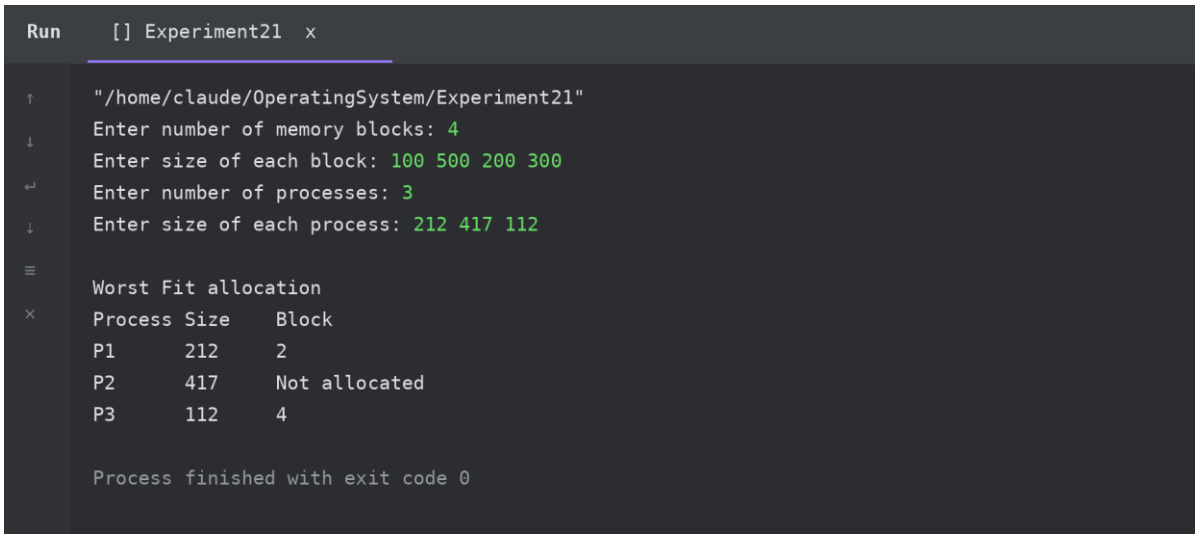
int main() {
    int nb, np, b[10], p[10], alloc[10], i, j, idx;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block: ");
    for (i = 0; i < nb; i++) scanf("%d", &b[i]);
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process: ");
    for (i = 0; i < np; i++) scanf("%d", &p[i]);

    for (i = 0; i < np; i++) {
        idx = -1;
        for (j = 0; j < nb; j++) {
            if (b[j] >= p[i]) {
                if (idx == -1 || b[j] > b[idx]) idx = j;     // largest block
that fits
            }
        }
        if (idx != -1) {
            alloc[i] = idx + 1;
            b[idx] -= p[i];             // remaining part of the block stays free
        } else {
            alloc[i] = 0;
        }
    }

    printf("\nWorst Fit allocation\n");
    printf("Process\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        if (alloc[i]) printf("P%d\t%d\t%d\n", i + 1, p[i], alloc[i]);
        else          printf("P%d\t%d\tNot allocated\n", i + 1, p[i]);
    }
    return 0;
}
```

Output Screenshot:



Experiment 22

Construct a C program to implement best fit algorithm of memory management.

Executable C code :

```
#include <stdio.h>

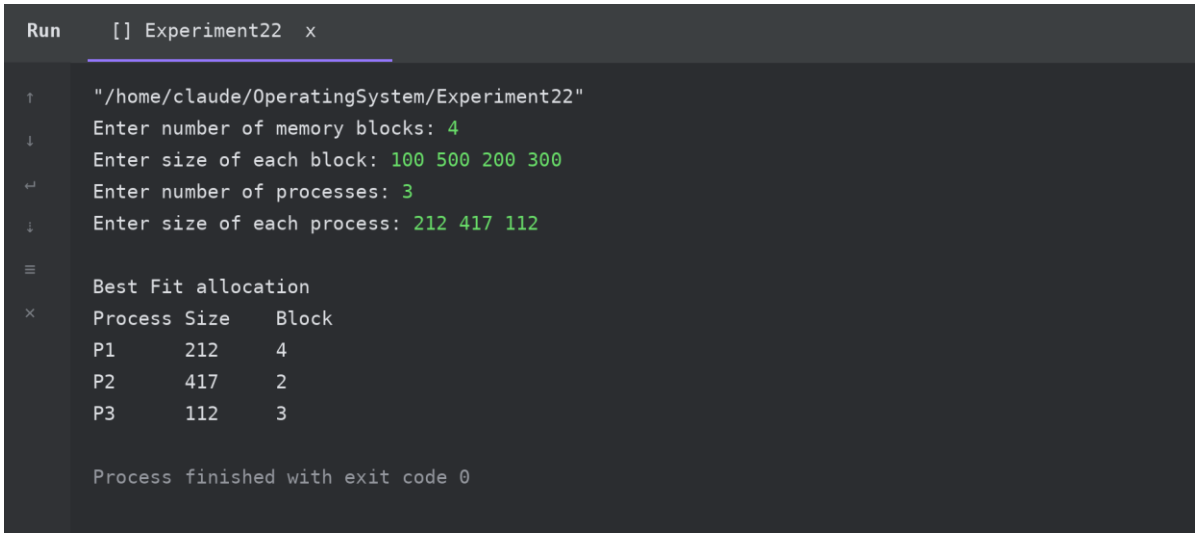
int main() {
    int nb, np, b[10], p[10], alloc[10], i, j, idx;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block: ");
    for (i = 0; i < nb; i++) scanf("%d", &b[i]);
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process: ");
    for (i = 0; i < np; i++) scanf("%d", &p[i]);

    for (i = 0; i < np; i++) {
        idx = -1;
        for (j = 0; j < nb; j++) {
            if (b[j] >= p[i]) {
                if (idx == -1 || b[j] < b[idx]) idx = j;     // smallest block
that fits
            }
        }
        if (idx != -1) {
            alloc[i] = idx + 1;
            b[idx] -= p[i];             // remaining part of the block stays free
        } else {
            alloc[i] = 0;
        }
    }

    printf("\nBest Fit allocation\n");
    printf("Process\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        if (alloc[i]) printf("P%d\t%d\t%d\n", i + 1, p[i], alloc[i]);
        else          printf("P%d\t%d\tNot allocated\n", i + 1, p[i]);
    }
    return 0;
}
```

Output Screenshot:



Experiment 23

Construct a C program to implement first fit algorithm of memory management.

Executable C code :

```
#include <stdio.h>

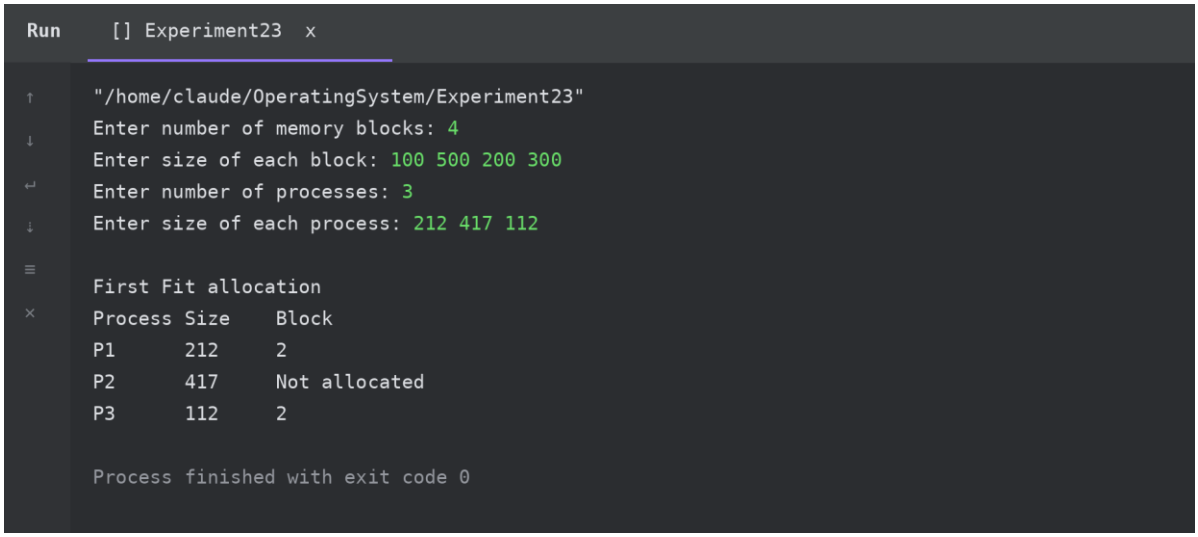
int main() {
    int nb, np, b[10], p[10], alloc[10], i, j, idx;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block: ");
    for (i = 0; i < nb; i++) scanf("%d", &b[i]);
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process: ");
    for (i = 0; i < np; i++) scanf("%d", &p[i]);

    for (i = 0; i < np; i++) {
        idx = -1;
        for (j = 0; j < nb; j++) {
            if (b[j] >= p[i]) {
                idx = j;                                    // first block that
fits
                break;
            }
        }
        if (idx != -1) {
            alloc[i] = idx + 1;
            b[idx] -= p[i];             // remaining part of the block stays free
        } else {
            alloc[i] = 0;
        }
    }

    printf("\nFirst Fit allocation\n");
    printf("Process\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        if (alloc[i]) printf("P%d\t%d\t%d\n", i + 1, p[i], alloc[i]);
        else          printf("P%d\t%d\tNot allocated\n", i + 1, p[i]);
    }
    return 0;
}
```

Output Screenshot:



Experiment 24

Design a C program to demonstrate UNIX system calls for file management.

Executable C code :

```c
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char text[100], buf[100];
    ssize_t n;
    off_t size;

    printf("Enter a word to write into the file: ");
    scanf("%s", text);

    fd = open("sample.txt", O_CREAT | O_RDWR | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    printf("open()   : sample.txt opened, file descriptor = %d\n", fd);

    n = write(fd, text, strlen(text));
    printf("write()  : %zd bytes written\n", n);

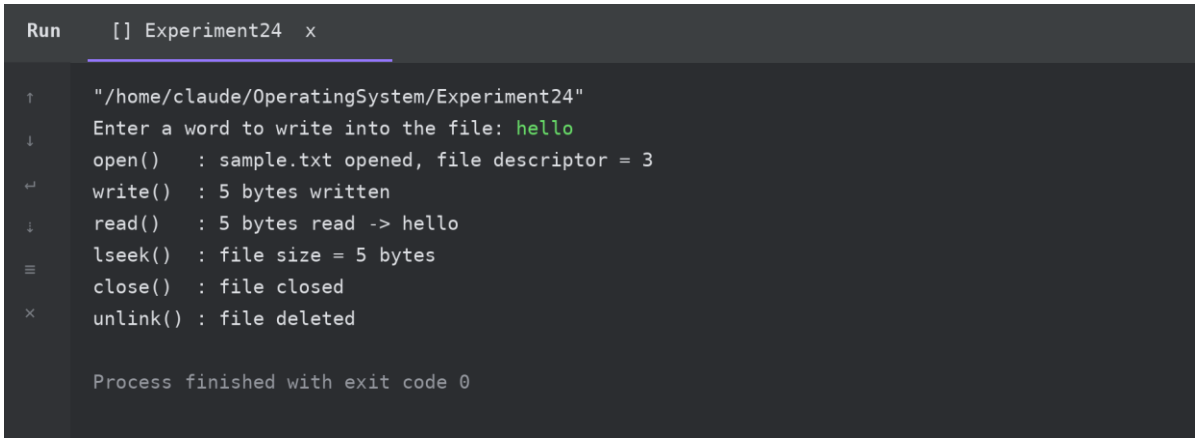
    lseek(fd, 0, SEEK_SET);                 // move back to the beginning
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read()   : %zd bytes read -> %s\n", n, buf);

    size = lseek(fd, 0, SEEK_END);          // offset of the end = file size
    printf("lseek()  : file size = %ld bytes\n", (long)size);

    close(fd);
    printf("close()  : file closed\n");

    unlink("sample.txt");
    printf("unlink() : file deleted\n");
    return 0;
}
```

Output Screenshot:



Experiment 25

Construct a C program to implement the I/O system calls of UNIX (fcntl, seek, stat, opendir, readdir)

Executable C code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>
#include <sys/stat.h>

int cmp(const void *a, const void *b) {
    return strcmp((const char *)a, (const char *)b);
}

int main() {
    char fname[50], path[100], names[20][256];
    int fd, flags, count = 0, i;
    struct stat st;
    DIR *dp;
    struct dirent *ent;

    printf("Enter a file name: ");
    scanf("%s", fname);

    mkdir("demo_dir", 0755);
    sprintf(path, "demo_dir/%s", fname);
    fd = open(path, O_CREAT | O_RDWR | O_TRUNC, 0644);
    write(fd, "Operating System", 16);
    close(open("demo_dir/second.txt", O_CREAT | O_WRONLY, 0644));

    // fcntl : read the file status flags
    flags = fcntl(fd, F_GETFL);
    printf("\nfcntl()   : access mode = %s\n",
           (flags & O_ACCMODE) == O_RDWR ? "Read/Write" :
           (flags & O_ACCMODE) == O_WRONLY ? "Write only" : "Read only");

    // lseek : current position and moving it
    printf("lseek()   : current offset = %ld\n", (long)lseek(fd, 0, SEEK_CUR));
    printf("lseek()   : offset after SEEK_SET 10 = %ld\n", (long)lseek(fd, 10,
SEEK_SET));

    // stat : file information
    stat(path, &st);
    printf("stat()    : size = %ld bytes, permissions = %o, links = %ld\n",
           (long)st.st_size, st.st_mode & 0777, (long)st.st_nlink);

    // opendir / readdir : list the directory
    dp = opendir("demo_dir");
    while ((ent = readdir(dp)) != NULL) {
        if (ent->d_name[0] != '.') strcpy(names[count++], ent->d_name);
    }
    closedir(dp);
    qsort(names, count, 256, cmp);
    printf("readdir() : files in demo_dir ->");
    for (i = 0; i < count; i++) printf(" %s", names[i]);
    printf("\n");

    close(fd);
    // clean up the demo files
    unlink(path);
    unlink("demo_dir/second.txt");
    rmdir("demo_dir");
```

```
    return 0;
}
```

Output Screenshot:

